Chapter 2 - Control Flow, List, Tuples, Sets, Dictionaries and Arrays in Python

October 12, 2020

1 Control Flow, List, Tuples, Sets, Dictionaries and Arrays in Python

1.1 Python Conditions and If statements

Python supports the usual logical conditions from mathematics:

```
Equals:                    a == b
Not Equals:                a != b
Less than:                 a < b
Less than or equal to:     a <= b
Greater than:              a > b
Greater than or equal to: a >= b
```

These conditions can be used in several ways, most commonly in "if statements" and loops.

An "if statement" is written by using the `if` keyword.

The `elif` keyword is pythons way of saying "if the previous conditions were not true, then try this condition".

The `else` keyword catches anything which isn't caught by the preceding conditions.

The `elif` and `else` keywords are optional.

[15]:
```
# If statement:
a = 200
b = 33
if b > a:
  print("b is greater than a")
elif a == b:
  print("a and b are equal")
else:
  print("a is greater than b")
```

```
a is greater than b
```

[3]:
```
a = 200
b = 33
if b > a:
  print("b is greater than a")
else:
  print("b is not greater than a")
```

```
b is not greater than a
```

1.2 Short Hand If

If you have only one statement to execute, you can put it on the same line as the if statement.

[4]:
```
# One line if statement:
if a > b: print("a is greater than b")
```

```
a is greater than b
```

1.3 Short Hand If ... Else

If you have only one statement to execute, one for if, and one for else, you can put it all on the same line:

[5]:
```
# One line if else statement:
a = 2
b = 330
print("A") if a > b else print("B")
```

```
B
```

Note! **Variables do not need to be declared with any particular type and can even change type after they have been set.**

1.4 While Loop

In Python, all the statements indented by the same number of character spaces after a programming construct are considered to be part of a single block of code. Python uses indentation as its method of grouping statements.

[1]:
```
# prints Hello Geek 3 Times
count = 0
while (count < 3):
    count = count+1
    print("Hello H2C Students")
```

```
Hello H2C Students
Hello H2C Students
Hello H2C Students
```

1.5 For in Loop

In Python, there is no C style for loop, i.e., for (i=0; i<n; i++). There is "for in" loop which is similar to for each loop in other languages. It can be used to iterate over iterators and a range.

[2]:
```
# Iterating over a list
print("List Iteration")
l = ["kamk", "for", "students"]
for i in l:
    print(i)

# Iterating over a tuple (immutable)
print("\nTuple Iteration")
t = ("kamk", "for", "h2c students")
for i in t:
    print(i)

# Iterating over a String
print("\nString Iteration")
s = "Kamk"
for i in s :
    print(i)

# Iterating over dictionary
print("\nDictionary Iteration")
d = dict()
d['xyz'] = 123
d['abc'] = 345
for i in d :
    print("%s  %d" %(i, d[i]))
```

```
List Iteration
kamk
for
students

Tuple Iteration
kamk
for
h2c students

String Iteration
```

```
K
a
m
k

Dictionary Iteration
xyz  123
abc  345
```

1.6 Nested Loops

Python programming language allows to use one loop inside another loop. Following section shows few examples to illustrate the concept. A final note on loop nesting is that we can put any type of loop inside of any other type of loop. For example a for loop can be inside a while loop or vice versa.

[3]:
```
from __future__ import print_function
for i in range(1, 5):
    for j in range(i):
         print(i, end=' ')
    print()
```

```
1
2 2
3 3 3
4 4 4 4
```

1.7 Loop Control Statements

Loop control statements change execution from its normal sequence. When execution leaves a scope, all automatic objects that were created in that scope are destroyed. Python supports the following control statements.

1.7.1 Continue Statement

It returns the control to the beginning of the loop.

[10]:
```
# Prints all letters except 'e' and 'a'
for letter in 'me opiskellaan':
    if letter == 'e' or letter == 'a':
         continue
    print ('Current Letter :', letter)
    var = 10
```

```
Current Letter : m
Current Letter :
Current Letter : o
Current Letter : p
Current Letter : i
Current Letter : s
Current Letter : k
Current Letter : l
Current Letter : l
Current Letter : n
```

1.8 Break Statement

It brings control out of the loop

[12]:
```
for letter in 'me opiskellaan':

    # break the loop as soon it sees 'e'
    # or 's'
    if letter == 'e' or letter == 'a':
         break

print ('Current Letter :', letter)
```

```
Current Letter : e
```

---

2 Python Data Types

2.1 Built-in Data Types

In programming, data type is an important concept.

Variables can store data of different types, and different types can do different things.

Python has the following data types built-in by default, in these categories:

Text Type: `str`

Numeric Types: `int`, `float`, `complex`

Sequence Types: `list`, `tuple`, `range`

Mapping Type: `dict`

Set Types: `set`, `frozenset`

Boolean Type: `bool`

Binary Types: `bytes`, `bytearray`, `memoryview`

---

3 Strings as arrays

Like many other popular programming languages, strings in Python are arrays of bytes representing unicode characters.

However, Python does not have a character data type, a single character is simply a string with a length of 1.

Square brackets can be used to access elements of the string.

[6]:
```
# Get the character at position 1 (remember that the first character has the␣
↪position 0):
a = "Hello, World!"
print(a[1])
```

```
e
```

3.1 Slicing

You can return a range of characters by using the slice syntax.

Specify the start index and the end index, separated by a colon, to return a part of the string.

[7]:
```
# Get the characters from position 2 to position 5 (not included):
b = "Hello, World!"
print(b[2:5])
```

```
llo
```

3.2 Negative Indexing

Use negative indexes to start the slice from the end of the string:

[8]:
```
# Get the characters from position 5 to position 1, starting the count from the␣
↪end of the string:
b = "Hello, World!"
print(b[-5:-2])
```

```
orl
```

3.3 String Length

To get the length of a string, use the len() function.

[9]:
```
# The len() function returns the length of a string:
a = "Hello, World!"
print(len(a))
```

```
13
```

3.4 String Methods

Python has a set of built-in methods that you can use on strings.

[10]:
```
# The strip() method removes any whitespace from the beginning or the end:
a = " Hello, World! "
print(a.strip()) # returns "Hello, World!"
```

```
Hello, World!
```

[11]:
```
# The lower() method returns the string in lower case:
a = "Hello, World!"
print(a.lower())
```

```
hello, world!
```

[12]:
```
# The upper() method returns the string in upper case:
a = "Hello, World!"
print(a.upper())
```

```
HELLO, WORLD!
```

[13]:
```
# The replace() method replaces a string with another string:
a = "Hello, World!"
print(a.replace("H", "J"))
```

```
Jello, World!
```

[14]:
```
# The split() method splits the string into substrings if it finds instances of␣
↪the separator:
a = "Hello, World!"
print(a.split(","))
```

```
['Hello', ' World!']
```

---

4 Python Casting

4.1 Specify a Variable Type

There may be times when you want to specify a type on to a variable. This can be done with casting. Python is an object-orientated language, and as such it uses classes to define data types, including its primitive types.

Casting in python is therefore done using constructor functions:

- `int()` - constructs an integer number from an integer literal, a float literal (by rounding down to the previous whole number), or a string literal (providing the string represents a whole number)
- `float()` - constructs a float number from an integer literal, a float literal or a string literal (providing the string represents a float or an integer)
- `str()` - constructs a string from a wide variety of data types, including strings, integer literals and float literals

[15]:
```
x = int(1)   # x will be 1
y = int(2.8) # y will be 2
z = int("3") # z will be 3
```

4.2 List

A list is a collection which is ordered and changeable. In Python lists are written with square brackets. Lists are just like dynamic sized arrays, declared in other languages (vector in C++ and ArrayList in Java). Lists need not be homogeneous always which makes it a most powerful tool in Python. A single list may contain DataTypes like Integers, Strings, as well as Objects. Lists are mutable, and hence, they can be altered even after their creation.

List in Python are ordered and have a definite count. The elements in a list are indexed according to a definite sequence and the indexing of a list is done with 0 being the first index. Each element in the list has its definite place in the list, which allows duplicating of elements in the list, with each element having its own distinct place and credibility.

Note! Lists are a useful tool for preserving a sequence of data and further iterating over it.

4.2.1 Creating a List

Lists in Python can be created by just placing the sequence inside the square brackets[].

[16]:
```
# Python program to demonstrate
# Creation of List

# Creating a List
List = []
print("Blank List: ")
print(List)
```

```
# Creating a List of numbers
List = [10, 20, 14]
print("\nList of numbers: ")
print(List)

# Creating a List of strings and accessing
# using index
List = ["Kamk", "H2C", "Students"]
print("\nList Items: ")
print(List[0])
print(List[2])

# Creating a Multi-Dimensional List
# (By Nesting a list inside a List)
List = [['Kamk', 'H2C'] , ['Students']]
print("\nMulti-Dimensional List: ")
print(List)
```

```
Blank List:
[]

List of numbers:
[10, 20, 14]

List Items:
Kamk
Students

Multi-Dimensional List:
[['Kamk', 'H2C'], ['Students']]
```

4.2.2 Creating a list with multiple distinct or duplicate elements

A list may contain duplicate values with their distinct positions and hence, multiple distinct or duplicate values can be passed as a sequence at the time of list creation.

[17]:
```
# Creating a List with
# the use of Numbers
# (Having duplicate values)
List = [1, 2, 4, 4, 3, 3, 3, 6, 5]
print("\nList with the use of Numbers: ")
print(List)

# Creating a List with
```

```
# mixed type of values
# (Having numbers and strings)
List = [1, 2, 'Kamk', 4, 'H2C', 6, 'Students']
print("\nList with the use of Mixed Values: ")
print(List)
```

```
List with the use of Numbers:
[1, 2, 4, 4, 3, 3, 3, 6, 5]

List with the use of Mixed Values:
[1, 2, 'Kamk', 4, 'H2C', 6, 'Students']
```

4.2.3 Knowing the size of List

[18]:
```
# Creating a List
List1 = []
print(len(List1))

# Creating a List of numbers
List2 = [10, 20, 14]
print(len(List2))
```

```
0
3
```

4.2.4 Adding Elements to a List

Using **append()** method

Elements can be added to the List by using built-in append() function. Only one element at a time can be added to the list by using append() method, for addition of multiple elements with the append() method, loops are used. Tuples can also be added to the List with the use of append method because tuples are immutable. Unlike Sets, Lists can also be added to the existing list with the use of append() method.

[20]:
```
# Python program to demonstrate
# Addition of elements in a List

# Creating a List
List = []
print("Initial blank List: ")
print(List)
```

```
# Addition of Elements
# in the List
List.append(1)
List.append(2)
List.append(4)
print("\nList after Addition of Three elements: ")
print(List)

# Adding elements to the List
# using Iterator
for i in range(1, 4):
    List.append(i)
print("\nList after Addition of elements from 1-3: ")
print(List)

# Adding Tuples to the List
List.append((5, 6))
print("\nList after Addition of a Tuple: ")
print(List)

# Addition of List to a List
List2 = ['H2C', 'Students']
List.append(List2)
print("\nList after Addition of a List: ")
print(List)
```

```
Initial blank List:
[]

List after Addition of Three elements:
[1, 2, 4]

List after Addition of elements from 1-3:
[1, 2, 4, 1, 2, 3]

List after Addition of a Tuple:
[1, 2, 4, 1, 2, 3, (5, 6)]

List after Addition of a List:
[1, 2, 4, 1, 2, 3, (5, 6), ['H2C', 'Students']]
```

4.2.5 Using insert() method

append() method only works for addition of elements at the end of the List, for addition of element at the desired position, insert() method is used. Unlike append() which takes only one argument, insert() method requires two arguments(position, value).

[23]:
```
# Python program to demonstrate
# Addition of elements in a List

# Creating a List
List = [1,2,3,4]
print("Initial List: ")
print(List)

# Addition of Element at
# specific Position
# (using Insert Method)
List.insert(3, 12) #Set 12 index 3
List.insert(0, 'Students') ## set index 0
print("\nList after performing Insert Operation: ")
print(List)
```

```
Initial List:
[1, 2, 3, 4]

List after performing Insert Operation:
['Students', 1, 2, 3, 12, 4]
```

4.2.6 Using extend() method

Other than append() and insert() methods, there's one more method for Addition of elements, extend(), this method is used to add multiple elements at the same time at the end of the list.

Note – append() and extend() methods can only add elements at the end.

[26]:
```
# Python program to demonstrate
# Addition of elements in a List

# Creating a List
List = [1,2,3,4]
print("Initial List: ")
print(List)

# Addition of multiple elements
# to the List at the end
# (using Extend Method)
```

```
List.extend([8, 'Students', 'Happy coding']) #add these three to the end of list
print("\nList after performing Extend Operation: ")
print(List)
```

```
Initial List:
[1, 2, 3, 4]

List after performing Extend Operation:
[1, 2, 3, 4, 8, 'Students', 'Happy coding']
```

4.2.7 Removing Elements from the List

Using remove() method

Elements can be removed from the List by using built-in remove() function but an Error arises if element doesn't exist in the set. Remove() method only removes one element at a time, to remove range of elements, iterator is used. The remove() method removes the specified item.

Note! Remove method in List will only remove the first occurrence of the searched element.

[27]:
```
# Python program to demonstrate
# Removal of elements in a List

# Creating a List
List = [1, 2, 3, 4, 5, 6,
        7, 8, 9, 10, 11, 12]
print("Intial List: ")
print(List)

# Removing elements from List
# using Remove() method
List.remove(5)
List.remove(6)
print("\nList after Removal of two elements: ")
print(List)

# Removing elements from List
# using iterator method
for i in range(1, 5):
    List.remove(i)
print("\nList after Removing a range of elements: ")
print(List)
```

```
Intial List:
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]

List after Removal of two elements:
```

```
[1, 2, 3, 4, 7, 8, 9, 10, 11, 12]

List after Removing a range of elements:
[7, 8, 9, 10, 11, 12]
```

4.2.8 Slicing of a List

In Python List, there are multiple ways to print the whole List with all the elements, but to print a specific range of elements from the list, we use Slice operation. Slice operation is performed on Lists with the use of a colon(:). To print elements from beginning to a range use [: Index], to print elements from end-use [:-Index], to print elements from specific Index till the end use [Index:], to print elements within a range, use [Start Index:End Index] and to print the whole List with the use of slicing operation, use [:]. Further, to print the whole List in reverse order, use [::-1].

Note – To print elements of List from rear end, use Negative Indexes.

If L is a list, the expression L [start : stop : step] returns the portion of the list from index start to index stop, at a step size step.

[28]:
```
#Here is a basic example of list slicing.

L = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i']
print(L[2:7])
# Prints ['c', 'd', 'e', 'f', 'g']
```

```
['c', 'd', 'e', 'f', 'g']
```

[ ]:
```
---
### List Methods
Function         Description
Append()         Add an element to the end of the list
Extend()         Add all elements of a list to the another list
Insert()         Insert an item at the defined index
Remove()         Removes an item from the list
Pop()          Removes and returns an element at the given index
Clear()          Removes all items from the list
Index()          Returns the index of the first matched item
Count()          Returns the count of number of items passed as an argument
Sort()          Sort items in a list in ascending order
Reverse()          Reverse the order of items in the list
copy()          Returns a copy of the list
```

4.3 Change Item Value

To change the value of a specific item, refer to the index number:

[30]:
```
# Change the second item:
thislist = ["apple", "banana", "cherry"]
thislist[1] = "blackcurrant"
print(thislist)
```

```
['apple', 'blackcurrant', 'cherry']
```

4.4 Loop Through a List

You can loop through the list items by using a for loop: Example:

[31]:
```
# Print all items in the list, one by one:
thislist = ["apple", "banana", "cherry"]
for x in thislist:
  print(x)
```

```
apple
banana
cherry
```

4.5 Check if Item Exists

To determine if a specified item is present in a list use the in keyword:

[32]:
```
# Check if "apple" is present in the list:
thislist = ["apple", "banana", "cherry"]
if "apple" in thislist:
  print("Yes, 'apple' is in the fruits list")
```

```
Yes, 'apple' is in the fruits list
```

4.6 Copy a List

You cannot copy a list simply by typing list2 = list1, because: list2 will only be a reference to list1, and changes made in list1 will automatically also be made in list2.

There are ways to make a copy, one way is to use the built-in List method copy().

[33]:
```
# Make a copy of a list with the copy() method:
thislist = ["apple", "banana", "cherry"]
mylist = thislist.copy()
print(mylist)
```

```
['apple', 'banana', 'cherry']
```

Another way to make a copy is to use the built-in method list().

[34]:
```
# Make a copy of a list with the list() method:
thislist = ["apple", "banana", "cherry"]
mylist = list(thislist)
print(mylist)
```

```
['apple', 'banana', 'cherry']
```

4.7 Join Two Lists

There are several ways to join, or concatenate, two or more lists in Python.

One of the easiest ways are by using the + operator.

[35]:
```
# Join two list:
list1 = ["a", "b" , "c"]
list2 = [1, 2, 3]

list3 = list1 + list2
print(list3)
```

```
['a', 'b', 'c', 1, 2, 3]
```

Another way to join two lists are by appending all the items from list2 into list1, one by one:

[36]:
```
# Append list2 into list1:
list1 = ["a", "b" , "c"]
list2 = [1, 2, 3]

for x in list2:
  list1.append(x)

print(list1)
```

```
['a', 'b', 'c', 1, 2, 3]
```

Or you can use the extend() method, which purpose is to add elements from one list to another list:

[37]:
```
# Use the extend() method to add list2 at the end of list1:
list1 = ["a", "b" , "c"]
list2 = [1, 2, 3]

list1.extend(list2)
print(list1)
```

```
['a', 'b', 'c', 1, 2, 3]
```

4.8 The list() Constructor

It is also possible to use the list() constructor to make a new list.

[38]:
```
# Using the list() constructor to make a List:
thislist = list(("apple", "banana", "cherry")) # note the double round-brackets
print(thislist)
```

```
['apple', 'banana', 'cherry']
```

4.9 Python List reverse()

The reverse() method reverses the elements of the list. The syntax of the reverse() method is:

```
list.reverse()
```

The reverse() method doesn't take any arguments. The reverse() method doesn't return any value. It updates the existing list.

[14]:
```
# Popular auto
auto = ['Volvo', 'Toyota', 'Opel', 'Mercedes']
print('Original List:', auto)

# List Reverse
auto.reverse()

# updated list
print('Updated List:', auto)
```

```
Original List: ['Volvo', 'Toyota', 'Opel', 'Mercedes']
Updated List: ['Mercedes', 'Opel', 'Toyota', 'Volvo']
```

---

5 Python Tuples

A tuple is a collection which is ordered and unchangeable. In Python tuples are written with round brack. Tuple is a collection of Python objects much like a list. The sequence of values stored in a tuple can be of any type, and they are indexed by integers. Values of a tuple are syntactically separated by 'commas'. Although it is not necessary, it is more common to define a tuple by closing the sequence of values in parentheses. This helps in understanding the Python tuples more easily. Creating a Tuple

In Python, tuples are created by placing sequence of values separated by 'comma' with or without the use of parentheses for grouping of data sequence.

Note! Creation of Python tuple without the use of parentheses is known as Tuple Packing.

[40]:
```
#Python program to demonstrate addition of elements in a Tuple.

#Creating an empty Tuple
Tuple1 = ()
print("Initial empty Tuple: ")
print (Tuple1)

#Creatting a Tuple
#with the use of string
Tuple1 = ('Kamk', 'H2C')
print("\nTuple with the use of String: ")
print(Tuple1)

# Creating a Tuple with
# the use of list
list1 = [1, 2, 4, 5, 6]
print("\nTuple using List: ")
print(tuple(list1))

#Creating a Tuple
#with the use of built-in function
Tuple1 = tuple('Students')
print("\nTuple with the use of function: ")
print(Tuple1)
```

```
Initial empty Tuple:
()

Tuple with the use of String:
('Kamk', 'H2C')

Tuple using List:
(1, 2, 4, 5, 6)

Tuple with the use of function:
```

```
('S', 't', 'u', 'd', 'e', 'n', 't', 's')
```

Tuples can contain any number of elements and of any datatype (like strings, integers, list, etc.). Tuples can also be created with a single element, but it is a bit tricky. Having one element in the parentheses is not sufficient, there must be a trailing 'comma' to make it a tuple.

[43]:
```
#Creating a Tuple
#with Mixed Datatype
Tuple1 = (5, 'Welcome all', 51, 'Students')
print("\nTuple with Mixed Datatypes: ")
print(Tuple1)

#Creating a Tuple
#with nested tuples
Tuple1 = (0, 1, 2, 3)
Tuple2 = ('to Learn Coding', 'Python')
Tuple3 = (Tuple1, Tuple2)
print("\nTuple with nested tuples: ")
print(Tuple3)

#Creating a Tuple
#with repetition
Tuple1 = ('Kamk',) * 3
print("\nTuple with repetition: ")
print(Tuple1)

#Creating a Tuple
#with the use of loop
Tuple1 = ('H2C Course')
n = 5
print("\nTuple with a loop")
for i in range(int(n)):
    Tuple1 = (Tuple1,)
    print(Tuple1)
```

```
Tuple with Mixed Datatypes:
(5, 'Welcome all', 51, 'Students')

Tuple with nested tuples:
((0, 1, 2, 3), ('to Learn Coding', 'Python'))

Tuple with repetition:
('Kamk', 'Kamk', 'Kamk')

Tuple with a loop
('H2C Course',)
(('H2C Course',),)
```

```
((('H2C Course',),),)
((((('H2C Course',),),),),)
(((((('H2C Course',),),),),),)
```

5.1 Change Tuple Values

Once a tuple is created, you cannot change its values. Tuples are unchangeable, or immutable as it also is called.

But there is a workaround. You can convert the tuple into a list, change the list, and convert the list back into a tuple.

[46]:
```
# Convert the tuple into a list to be able to change it:
x = ("apple", "banana", "cherry")
y = list(x)
y[1] = "kiwi" #set new value to index 1
x = tuple(y)

print(x)
```

```
('apple', 'kiwi', 'cherry')
```

5.2 Create Tuple With One Item

To create a tuple with only one item, you have to add a comma after the item, otherwise Python will not recognize it as a tuple.

[47]:
```
# One item tuple, remember the commma:
thistuple = ("apple",)
print(type(thistuple))
#NOT a tuple
thistuple = ("apple")
print(type(thistuple))
```

```
<class 'tuple'>
<class 'str'>
```

5.3 Concatenation of Tuples - Join Two Tuples

Concatenation of tuple is the process of joining of two or more Tuples. Concatenation is done by the use of '+' operator. Concatenation of tuples is done always from the end of the original tuple.

Other arithmetic operations do not apply on Tuples. Note! Only same datatypes can be combined with concatenation, an error arises if a list and a tuple are combined.

[48]:
```
# Concatenaton of tuples
Tuple1 = (0, 1, 2, 3)
Tuple2 = ('Kamk', 'H2C', 'Students')

Tuple3 = Tuple1 + Tuple2

# Printing first Tuple
print("Tuple 1: ")
print(Tuple1)

# Printing Second Tuple
print("\nTuple2: ")
print(Tuple2)

# Printing Final Tuple
print("\nTuples after Concatenaton: ")
print(Tuple3)
```

```
Tuple 1:
(0, 1, 2, 3)

Tuple2:
('Kamk', 'H2C', 'Students')

Tuples after Concatenaton:
(0, 1, 2, 3, 'Kamk', 'H2C', 'Students')
```

---

6 Python Sets

In Python, Set is an unordered collection of data type that is iterable, mutable and has no duplicate elements. The order of elements in a set is undefined though it may consist of various elements.

The major advantage of using a set, as opposed to a list, is that it has a highly optimized method for checking whether a specific element is contained in the set.

6.0.1 Creating a Set

Sets can be created by using the built-in set() function with an iterable object or a sequence by placing the sequence inside curly braces, separated by 'comma'.

Note! A set cannot have mutable elements like a list, set or dictionary, as its elements.

[50]:
```
# Python program to demonstrate
# Creation of Set in Python

# Creating a Set
set1 = set()
print("Intial blank Set: ")
print(set1)

# Creating a Set with
# the use of a String
set1 = set("Kajaanin ammattikorkeakoulu")
print("\nSet with the use of String: ")
print(set1)

# Creating a Set with
# the use of Constructor
# (Using object to Store String)
String = 'Kajaanin ammattikorkeakoulu'
set1 = set(String)
print("\nSet with the use of an Object: " )
print(set1)

# Creating a Set with
# the use of a List
set1 = set(["Kajaanin", "Amk", "Kamk"])
print("\nSet with the use of List: ")
print(set1)
```

```
Intial blank Set:
set()

Set with the use of String:
{'n', 'e', 'j', 'a', 't', 'r', 'i', 'K', 'o', 'u', 'm', 'k', 'l', ' '}

Set with the use of an Object:
{'n', 'e', 'j', 'a', 't', 'r', 'i', 'K', 'o', 'u', 'm', 'k', 'l', ' '}

Set with the use of List:
{'Kajaanin', 'Kamk', 'Amk'}
```

A set contains only unique elements but at the time of set creation, multiple duplicate values can also be passed. Order of elements in a set is undefined and is unchangeable. Type of elements in a set need not be the same, various mixed up data type values can also be passed to the set.

---

7 Python Dictionaries

Dictionary in Python is an unordered collection of data values, used to store data values like a map, which unlike other Data Types that hold only single value as an element, Dictionary holds key:value pair. Key value is provided in the dictionary to make it more optimized.

Note – Keys in a dictionary doesn't allows Polymorphism.

7.0.1 Creating a Dictionary

In Python, a Dictionary can be created by placing sequence of elements within curly {} braces, separated by 'comma'. Dictionary holds a pair of values, one being the Key and the other corresponding pair element being its Key:value. Values in a dictionary can be of any datatype and can be duplicated, whereas keys can't be repeated and must be immutable.

Note! Dictionary keys are case sensitive, same name but different cases of Key will be treated distinctly.

[51]:
```
# Creating a Dictionary
# with Integer Keys
Dict = {1: 'Kamk', 2: 'For', 3: 'H2C'}
print("\nDictionary with the use of Integer Keys: ")
print(Dict)

# Creating a Dictionary
# with Mixed keys
Dict = {'Name': 'Kamk', 1: [1, 2, 3, 4]}
print("\nDictionary with the use of Mixed Keys: ")
print(Dict)
```

```
Dictionary with the use of Integer Keys:
{1: 'Kamk', 2: 'For', 3: 'H2C'}

Dictionary with the use of Mixed Keys:
{'Name': 'Kamk', 1: [1, 2, 3, 4]}
```

Dictionary can also be created by the built-in function dict(). An empty dictionary can be created by just placing to **curly braces{}**.

[52]:
```
# Creating an empty Dictionary
Dict = {}
print("Empty Dictionary: ")
print(Dict)

# Creating a Dictionary
# with dict() method
Dict = dict({1: 'Kamk', 2: 'For', 3:'H2C'})
```

```
print("\nDictionary with the use of dict(): ")
print(Dict)

# Creating a Dictionary
# with each item as a Pair
Dict = dict([(1, 'Kamk'), (2, 'For')])
print("\nDictionary with each item as a pair: ")
print(Dict)
```

```
Empty Dictionary:
{}

Dictionary with the use of dict():
{1: 'Kamk', 2: 'For', 3: 'H2C'}

Dictionary with each item as a pair:
{1: 'Kamk', 2: 'For'}
```

7.0.2 Nested Dictionary - use key Value set pairs

Keys[1]: Value Set['Kamk'] 3:{'A' : 'H2C' Here 'A' is Nested keys and 'H2C' is a Value Set and so on...

[55]:
```
# Creating a Nested Dictionary
# as shown in the below image
Dict = {1: 'Kamk', 2: 'For',
        3:{'A' : 'H2C', 'B' : 'To', 'C' : 'Students'}}

print(Dict)
```

```
{1: 'Kamk', 2: 'For', 3: {'A': 'H2C', 'B': 'To', 'C': 'Students'}}
```

7.0.3 Adding elements to a Dictionary

In Python Dictionary, Addition of elements can be done in multiple ways. One value at a time can be added to a Dictionary by defining value along with the key e.g. Dict[Key] = 'Value'. Updating an existing value in a Dictionary can be done by using the built-in update() method. Nested key values can also be added to an existing Dictionary.

Note! While adding a value, if the key value already exists, the value gets updated otherwise a new Key with the value is added to the Dictionary.

[56]:
```
# Creating an empty Dictionary
Dict = {}
```

```
print("Empty Dictionary: ")
print(Dict)

# Adding elements one at a time
Dict[0] = 'Kamk'
Dict[2] = 'For'
Dict[3] = 1
print("\nDictionary after adding 3 elements: ")
print(Dict)

# Adding set of values
# to a single Key
Dict['Value_set'] = 2, 3, 4
print("\nDictionary after adding 3 elements: ")
print(Dict)

# Updating existing Key's Value
Dict[2] = 'Welcome'
print("\nUpdated key value: ")
print(Dict)

# Adding Nested Key value to Dictionary
Dict[5] = {'Nested' :{'1' : 'enjoy', '2' : 'Python code'}}
print("\nAdding a Nested Key: ")
print(Dict)
```

```
Empty Dictionary:
{}

Dictionary after adding 3 elements:
{0: 'Kamk', 2: 'For', 3: 1}

Dictionary after adding 3 elements:
{0: 'Kamk', 2: 'For', 3: 1, 'Value_set': (2, 3, 4)}

Updated key value:
{0: 'Kamk', 2: 'Welcome', 3: 1, 'Value_set': (2, 3, 4)}

Adding a Nested Key:
{0: 'Kamk', 2: 'Welcome', 3: 1, 'Value_set': (2, 3, 4), 5: {'Nested': {'1':
'enjoy', '2': 'Python code'}}}
```

7.1 Removing items

[63]:
```
# Creating an empty Dictionary
Dict = {}
print("Empty Dictionary: ")
print(Dict)

# Adding elements one at a time
Dict[0] = 'Kamk'
Dict[2] = 'For'
Dict[3] = 1
print("\nDictionary after adding 3 elements: ")
print(Dict)

# Adding set of values
# to a single Key
Dict['Value_set'] = 2, 3, 4
print("\nDictionary after adding 3 elements: ")
print(Dict)

# Updating existing Key's Value
Dict[2] = 'Welcome'
print("\nUpdated key value: ")
print(Dict)

# Adding Nested Key value to Dictionary
Dict[5] = {'Nested' :{'1' : 'enjoy', '2' : 'Python code'}}
print("\nAdding a Nested Key: ")
print(Dict)

# Pop element
Popping_element = Dict[5].pop('Nested')
print(Dict)
print('Value associated to poped key is: ' + str(Popping_element))
```

```
Empty Dictionary:
{}

Dictionary after adding 3 elements:
{0: 'Kamk', 2: 'For', 3: 1}

Dictionary after adding 3 elements:
{0: 'Kamk', 2: 'For', 3: 1, 'Value_set': (2, 3, 4)}

Updated key value:
{0: 'Kamk', 2: 'Welcome', 3: 1, 'Value_set': (2, 3, 4)}
```

```
Adding a Nested Key:
{0: 'Kamk', 2: 'Welcome', 3: 1, 'Value_set': (2, 3, 4), 5: {'Nested': {'1':
'enjoy', '2': 'Python code'}}}
{0: 'Kamk', 2: 'Welcome', 3: 1, 'Value_set': (2, 3, 4), 5: {}}
Value associated to poped key is: {'1': 'enjoy', '2': 'Python code'}
```

---

8 Python Arrays

An array is a collection of items stored at contiguous memory locations. The idea is to store multiple items of the same type together. This makes it easier to calculate the position of each element by simply adding an offset to a base value, i.e., the memory location of the first element of the array (generally denoted by the name of the array).

For simplicity, we can think of an array a fleet of stairs where on each step is placed a value (let's say one of your friends). Here, you can identify the location of any of your friends by simply knowing the count of the step they are on. Array can be handled in Python by a module named array. They can be useful when we have to manipulate only a specific data type values. A user can treat lists as arrays. However, user cannot constraint the type of elements stored in a list. If you create arrays using the array module, all elements of the array must be of the same type.

8.1 Creating a Array

Array in Python can be created by importing array module. array(data_type, value_list) is used to create an array with data type and value list specified in its arguments.

[16]:
```python
# Python program to demonstrate
# Creation of Array

# importing "array" for array creations
import array as arr

# creating an array with integer type
a = arr.array('i', [1, 2, 3])

# printing original array
print ("The new created array is : ", end =" ")
for i in range (0, 3):
    print (a[i], end =" ")
print()

# creating an array with float type
b = arr.array('d', [2.5, 3.2, 3.3])
```

```
# printing original array
print ("The new created array is : ", end =" ")
for i in range (0, 3):
    print (b[i], end =" ")
```

```
The new created array is :  1 2 3
The new created array is :  2.5 3.2 3.3
```

8.2 Adding Elements to a Array

Elements can be added to the Array by using built-in insert() function. Insert is used to insert one or more data elements into an array. Based on the requirement, a new element can be added at the beginning, end, or any given index of array. append() is also used to add the value mentioned in its arguments at the end of the array.

[20]:
```
# Python program to demonstrate
# Adding Elements to a Array

# importing "array" for array creations
import array as arr # Note these kind of syntax when we import some module and␣
 ↪its operation to be used inside code

# array with int type
a = arr.array('i', [1, 2, 3])


print ("Array before insertion : ", end =" ")
for i in range (0, 3):
    print (a[i], end =" ")
print()

# inserting array using
# insert() function
a.insert(1, 4)

print ("Array after insertion : ", end =" ")
for i in (a):
    print (i, end =" ")
print()

# array with float type
b = arr.array('d', [2.5, 3.2, 3.3])

print ("Array before insertion : ", end =" ")
for i in range (0, 3):
```

```
    print (b[i], end =" ")
print()

# adding an element using append()
b.append(4.4)

print ("Array after insertion : ", end =" ")
for i in (b):
    print (i, end =" ")
print()
```

```
Array before insertion :  1 2 3
Array after insertion :  1 4 2 3
Array before insertion :  2.5 3.2 3.3
Array after insertion :  2.5 3.2 3.3 4.4
```

8.3 Accessing elements from the Array

In order to access the array items refer to the index number. Use the index operator [ ] to access an item in a array. The index must be an integer.

[21]:
```
# Python program to demonstrate
# accessing of element from list

# importing array module
import array as arr

# array with int type - Note i which tells that array contais int type
a = arr.array('i', [1, 2, 3, 4, 5, 6])

# accessing element of array
print("Access element is: ", a[0])

# accessing element of array
print("Access element is: ", a[3])

# array with float type - d indicates this
b = arr.array('d', [2.5, 3.2, 3.3])

# accessing element of array
print("Access element is: ", b[1])

# accessing element of array
print("Access element is: ", b[2])
```

```
Access element is:  1
Access element is:  4
Access element is:  3.2
Access element is:  3.3
```

8.4 Removing Elements from the Array

Elements can be removed from the array by using built-in remove() function but an Error arises if element doesn't exist in the set. Remove() method only removes one element at a time, to remove range of elements, iterator is used. pop() function can also be used to remove and return an element from the array, but by default it removes only the last element of the array, to remove element from a specific position of the array, index of the element is passed as an argument to the pop() method.

Note – Remove method in List will only remove the first occurrence of the searched element

[22]:
```
# Python program to demonstrate
# Removal of elements in a Array

# importing "array" for array operations
import array

# initializing array with array values
# initializes array with signed integers
arr = array.array('i', [1, 2, 3, 1, 5])

# printing original array
print ("The new created array is : ", end ="")
for i in range (0, 5):
    print (arr[i], end =" ")

print ("\r")

# using pop() to remove element at 2nd position
print ("The popped element is : ", end ="")
print (arr.pop(2))

# printing array after popping
print ("The array after popping is : ", end ="")
for i in range (0, 4):
    print (arr[i], end =" ")

print("\r")

# using remove() to remove 1st occurrence of 1
arr.remove(1)
```

```
# printing array after removing
print ("The array after removing is : ", end ="")
for i in range (0, 3):
    print (arr[i], end =" ")
```

```
The new created array is : 1 2 3 1 5
The popped element is : 3
The array after popping is : 1 2 1 5
The array after removing is : 2 1 5
```

8.5 Searching element in a Array

In order to search an element in the array we use a python in-built index() method. This function returns the index of the first occurrence of value mentioned in arguments.

[27]:
```
# Python code to demonstrate
# searching an element in array


# importing array module
import array

# initializing array with array values
# initializes array with signed integers
arr = array.array('i', [1, 2, 3, 1, 2, 5])
array_len = len(arr)

# printing original array
print ("The new created array is : ", end ="")
for i in range (0, 6):
    print (arr[i], end =" ")

print ("\r")

# or using array_len
print ("The new created array using array_len variable : ", end ="")
for i in range (0, array_len):
    print (arr[i], end =" ")

print ("\r")

# using index() to print index of 1st occurrenece of 2
print ("The index of 1st occurrence of 2 is : ", end ="")
print (arr.index(2))
```

```
# using index() to print index of 1st occurrenece of 1
print ("The index of 1st occurrence of 1 is : ", end ="")
print (arr.index(1))
```

```
The new created array is : 1 2 3 1 2 5
The new created array using array_len variable : 1 2 3 1 2 5
The index of 1st occurrence of 2 is : 1
The index of 1st occurrence of 1 is : 0
```

8.6 Updating Elements in a Array

In order to update an element in the array we simply reassign a new value to the desired index we want to update.

[28]:
```
# Python code to demonstrate
# how to update an element in array

# importing array module
import array as arr

# initializing array with array values
# initializes array with signed integers
arr = array.array('i', [1, 2, 3, 1, 2, 5])

# printing original array
print ("Array before updation : ", end ="")
for i in range (0, 6):
    print (arr[i], end =" ")

print ("\r")

# updating a element in a array
arr[2] = 6
print("Array after updation : ", end ="")
for i in range (0, 6):
    print (arr[i], end =" ")
print()

# updating a element in a array
arr[4] = 8
print("Array after updation : ", end ="")
for i in range (0, 6):
    print (arr[i], end =" ")
```

```
Array before updation : 1 2 3 1 2 5
Array after updation : 1 2 6 1 2 5
```

```
Array after updation : 1 2 6 1 8 5
```

Get started learning Python with DataCamp's free Intro to Python tutorial. Learn Data Science by completing interactive coding challenges and watching videos by expert instructors. Link: https://www.learnpython.org/